

Explainable GNN for Research Group Prediction in the AIFB Dataset

Anmol Raj Srivastav [4064093] Janhavi Palav [4063279]

University of Paderborn, Germany

{anmolraj,pjanhavi}@campus.uni-paderborn.de

Abstract

This mini-project builds an explainable Graph Neural Network pipeline for node classification on the AIFB RDF knowledge graph. Given a person node, the task is to predict which of four research groups they belong to. We trained a two-layer FastRGCN model on top of a learnable node embedding table, since the dataset doesn't come with dense node features of its own. We chose FastRGCN deliberately over the plain RGCNConv layer. We received an accuracy of 91.67%.

With the trained model in hand, we applied GNNExplainer in two ways: a global pass that averages edge-mask importance by relation type over a sample of training nodes, and a local pass that produces a labelled subgraph explanation for five individual test-set people. We also evaluated these explanations quantitatively using Fidelity+, Fidelity-, Sparsity, and Stability. Across the board, the explanations came out sparse and highly stable across repeated runs, and in every case we tested, they were sufficient on their own to reproduce the model's prediction.

Keywords: Explainable Artificial Intelligence, Graph Neural Networks, RDF Knowledge Graphs, FastRGCN, GNNExplainer, AIFB

1 Introduction

Graph-structured data shows up naturally in knowledge graphs, citation networks, and social networks, where entities are connected by typed, directional relationships instead of being independent samples. Graph Neural Networks handle this by learning node representations through message passing, where each node repeatedly updates its representation based on its neighbours. The AIFB dataset we used here is an RDF knowledge graph describing staff, research groups, and publications at the AIFB research institute, and our task is to predict a person's research group affiliation just from the structure of the graph.

Because RDF relations are typed—**author** means something completely different from **phone** or **worksAtProject**—a relational GNN made much more sense for this than a plain homogeneous GCN. We used FastRGCN, PyTorch Geometric's implementation of Relational Graph Convolutional Networks. The choice of FastRGCN over the standard RGCNConv layer came down to a decisive reason that RGCNConv simply isn't compatible with GNNExplainer, while FastRGCNConv is, and since explainability was the whole point of this project, that settled it.

Like most deep models, GNNs don't come with any built-in way to explain their decisions. So we paired the FastRGCN classifier with GNNExplainer, which learns a soft importance mask over the edges around a given prediction. Our goal wasn't just to build a working classifier but it was to actually be able to point at the specific graph connections that pushed the model toward a particular research group.

2 Data Analysis

The dataset used in this project is the AIFB RDF dataset. It is a full AIFB graph which is distributed as an RDF file of triples. Loading it directly with RDFLib gives 29,226 triples. For the learning task we use PyTorch Geometric's built-in **Entities** loader for the AIFB dataset, which parses the raw n-triples file, strips out the predicates that would directly leak the target label, and adds an inverse edge for every relation so that messages can flow in both directions during message passing. This roughly doubles the number of distinct relation types and yields a processed graph of 8,285 nodes and 90 relation types.

Only a small subset of these nodes are person nodes with a known research-group label and everything else including publications, organisations, contact details, and so on exists purely to provide structural context.

Table 1 summarises the key statistics of the dataset after preprocessing.

Table 1: AIFB Dataset Statistics.

Statistic	Value
Total RDF triples	29,226
Unique nodes	8,285
Unique edge types	90
Train nodes	140
Test nodes	36

The graph contains 8,285 nodes which represents a variety of entities including persons, publications, topics, and organisations. Only a subset of these nodes corresponds to people with known research group affiliations, and the classification task focuses exclusively on these labelled nodes. The dataset contains 90 distinct edge types (45 original relations and their inverses added during preprocessing in order to enable bidirectional message passing).

Fig. 1 shows the four research groups and their member counts are:

- Business Information and Communication Systems: 73
- Knowledge Management: 60
- Efficient Algorithms: 28
- Complexity Management: 16

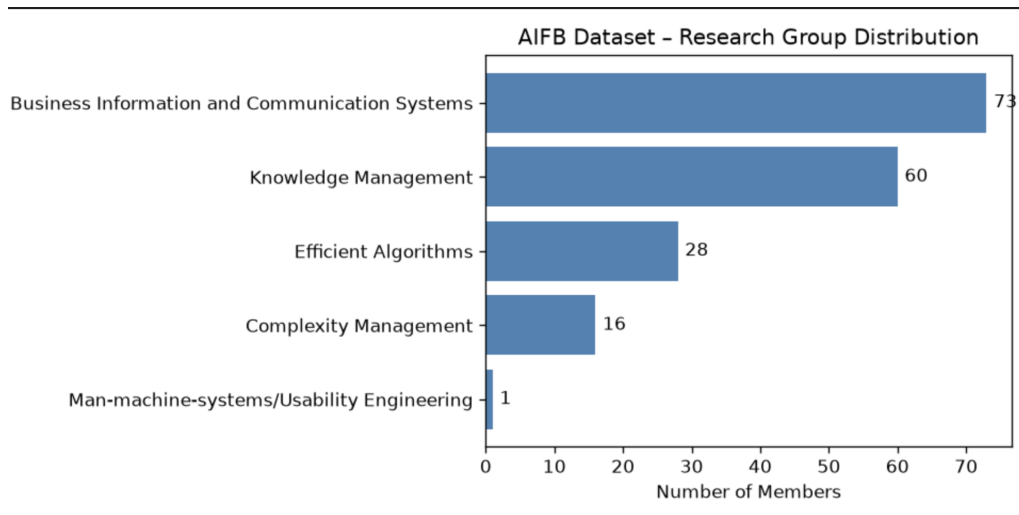


Figure 1: Distribution of individuals across the four research group classes in the AIFB dataset.

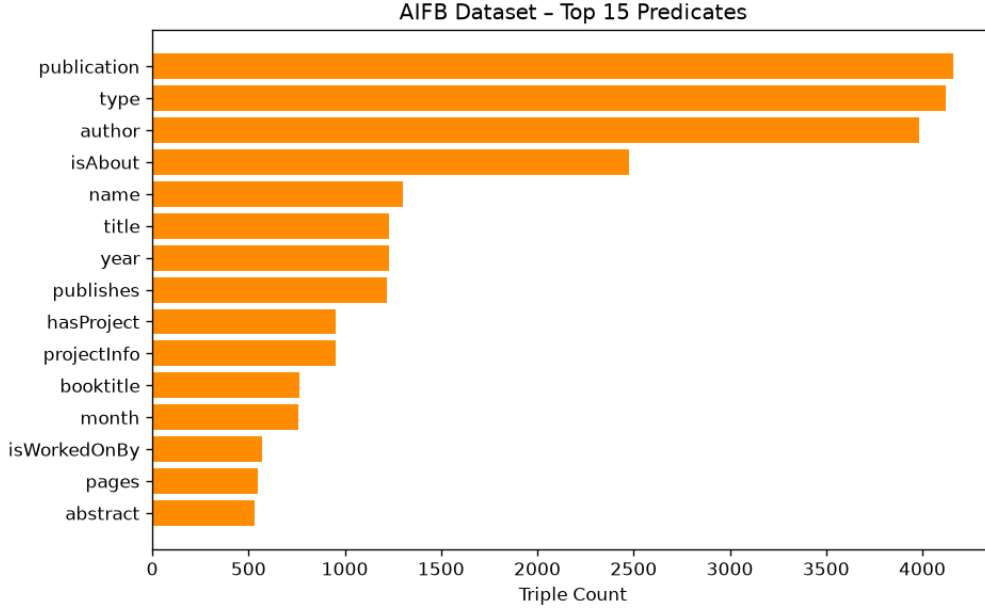


Figure 2: Top 15 most frequent predicates (edge types) in the AIFB RDF graph.

Fig. 2 shows the most frequent predicates in the graph. The most common relations include **publication**, **author**, and **type**. These relations connect persons to their publications, publication venues, and ontological types, providing rich structural signals for the classification task.

3 Model Training and Evaluation

3.1 Architecture

The AIFB dataset does not come with dense node features, so the model starts with a learnable embedding table: each of the 8,285 node ids is mapped to a 32-dimensional vector that is trained jointly with the rest of the network, rather than using a fixed one-hot encoding. These embeddings are passed through two FastRGCNConv layers. The first layer maps the 32-dimensional embedding down to a 16-dimensional hidden representation, followed by a ReLU activation and dropout. The second layer maps the hidden representation to four class logits, one per research group.

Table 2: FastRGCN model configuration.

Component	Setting
Input representation	Learnable node embedding
Embedding dimension	32
Hidden dimension	16
GNN layers	$2 \times$ FastRGCNConv
Output classes	4 research groups
Dropout	0.2
Optimizer	Adam
Learning rate	0.01
Weight decay	0.0005
Epochs per run	50
Loss	Cross-entropy loss

3.2 Training procedure

The AIFB test split only has 36 nodes, so a single training run’s accuracy can swing by roughly one class’s worth of nodes depending on the random seed alone – a couple of percentage points either way

is essentially noise rather than a meaningful difference in model quality. To avoid reporting a lucky (or unlucky) single run, `model_training.py` trains the same FastRGCN architecture from five different seeds (0, 1, 2, 42, 123), records the test accuracy and macro-F1 of each, and then keeps the model from the *median*-accuracy seed as the representative checkpoint used for all downstream explanation work. This avoids quietly cherry-picking whichever seed happened to score highest on the test set.

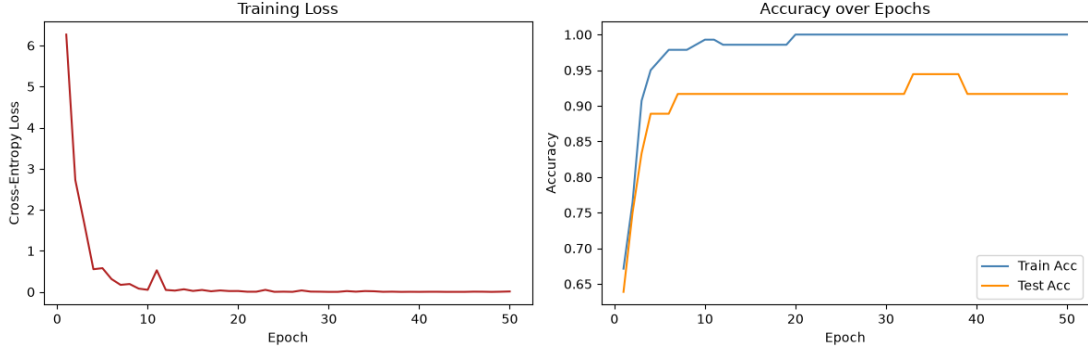


Figure 3: Training loss and accuracy curves over 50 epochs for the median-accuracy seed. The training loss drops sharply within the first 15–20 epochs and training accuracy saturates near 100%, while test accuracy climbs more gradually and settles at a high but visibly noisier level.

Across the five seeds, training accuracy reliably reaches 100%, which is unsurprising given how small the labelled set is relative to the model’s capacity. Test accuracy is more informative: across the five seeds the model reached a mean test accuracy of $92.2\% \pm 2.1$, with the representative median-seed model at 91.7% test accuracy and a macro-F1 of 0.88, and the errors that do occur concentrate in Complexity Management, the smallest of the four classes, which is exactly where the class imbalance shown in Figure Class Distribution would predict trouble.

4 Model Explanation

4.1 GNNExplainer setup

GNNExplainer works by learning a soft mask over the edges of the graph and optimising it so that the masked subgraph reproduces the model’s original prediction as closely as possible; edges with a high mask value are the ones the explainer considers necessary for that prediction. One practical complication is that our saved FastRGCN model begins with a learnable embedding lookup, and GNNExplainer expects to work with continuous node features rather than integer node indices. We handle this by first computing the embedding matrix once with the trained embedding layer, and then wrapping only the two trained FastRGCNConv layers in a small explainer-facing module (`FastRGCNExplainer`) that GNNExplainer can optimise against. The explanation is therefore still produced for the actual trained convolutional layers, not for some separate approximation of the model.

For both the global and local explanations we configure GNNExplainer with `edge_mask_type="object"` and `node_mask_type=None`, so the explainer focuses purely on which edges matter rather than reweighting node features as well.

4.2 Global explanation

The global explanation runs GNNExplainer on the first 20 training nodes, collects the resulting edge masks, and averages the importance score of every edge by its relation type. This gives a relation-level summary of what the model tends to rely on across the dataset as a whole, rather than for any one person.

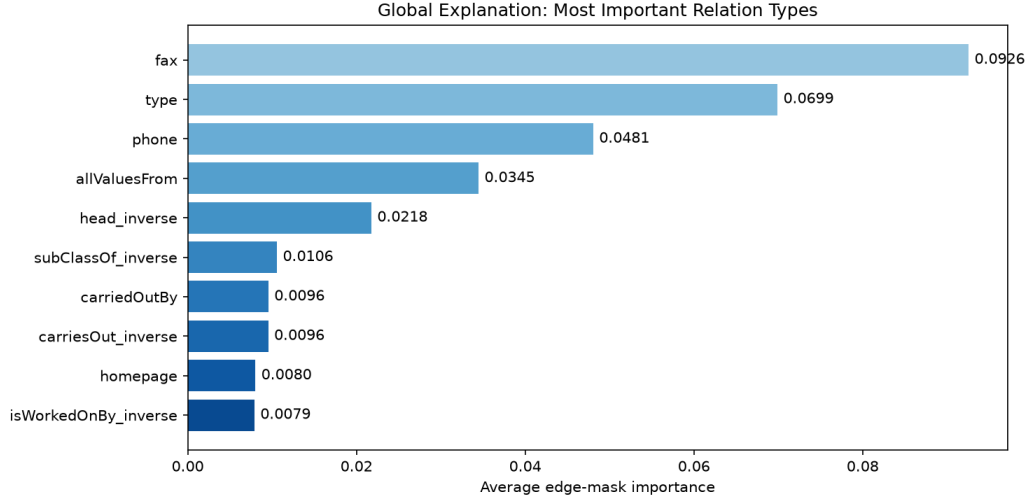


Figure 4: Global explanation: average GNNExplainer edge-mask importance per relation type, computed over 20 sampled training nodes.

Table 3: Top 10 relation types by average global importance.

Rank	Relation	Average importance
1	fax	0.0926
2	type	0.0699
3	phone	0.0481
4	allValuesFrom	0.0345
5	head_inverse	0.0218
6	subClassOf_inverse	0.0106
7	carriedOutBy	0.0096
8	carriesOut_inverse	0.0096
9	homepage	0.0080
10	isWorkedOnBy_inverse	0.0079

When looking at the list, it is a little surprising to see **fax** at the top since its just contact info and not an obviously meaningful academic feature. But this is more understandable in the context of the AIFB data set. Researchers of the same group would often share office contact information such as fax and phone numbers which will amke these relationships useful indicators of group membership. **Type** edges similarly differentiate whether a node is a Person, Project or Organisation, which adds additional structural information to help the model in identifying affiliations. More meaningful semantic relations also arise among the most important features, such as **carriedOutBy**, **carriesOut_inverse**, and **isWorkedOnBy_inverse** which connect researchers to the projects and research topics belonging to their groups.

4.3 Local Explanations

Local explanations were generated for five test-set persons who were correctly classified by the model, using 200 GNNExplainer optimisation epochs per person and restricting the reported edges to those inside the node’s 2-hop receptive field (the model only has two convolutional layers, so anything further away cannot actually influence its prediction).

Table 4: Summary of the five local person explanations.

Person	Predicted group	Confidence	Correct	Important edges
Andreas Oberweiss	Business Information & Communication Systems	1.000	Yes	2
Maik Herfurth	Business Information & Communication Systems	1.000	Yes	2
Sebastian Blohm	Knowledge Management	1.000	Yes	3
Peter Bungert	Efficient Algorithms	1.000	Yes	3
Saartje Brockmans	Knowledge Management	1.000	Yes	6

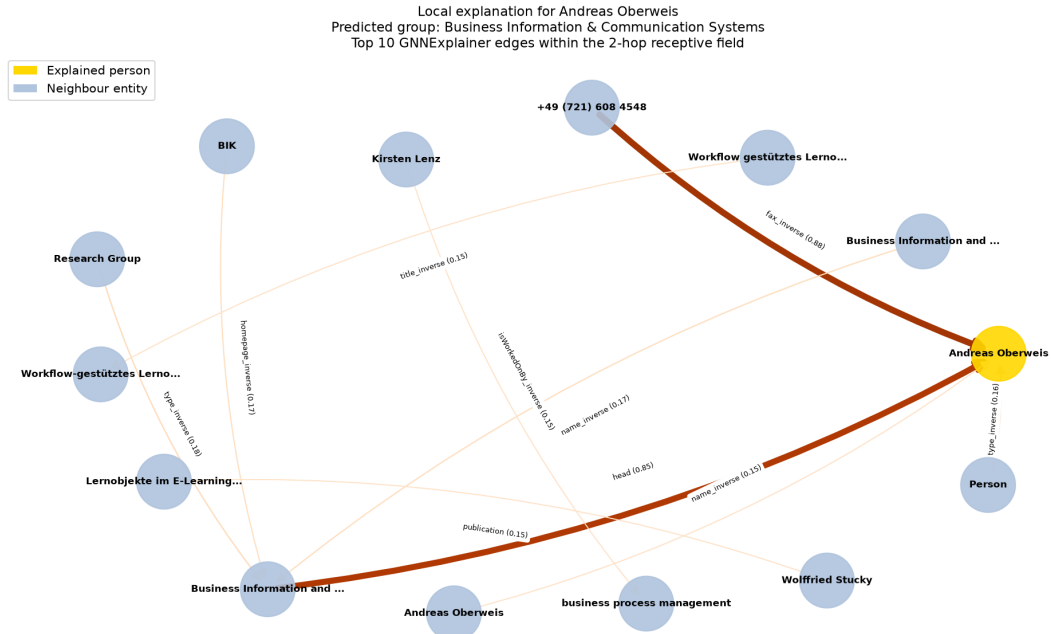


Figure 5: Local explanation subgraph for Andreas Oberweis. Gold marks the explained person; blue marks neighbouring entities pulled in through important edges.

For **Andreas Oberweis**, the model predicts Business Information and Communication Systems with full confidence. The strongest single edge is his fax number, closely followed by a direct **head** link from the research group node itself to Oberweis. The latter is about as direct a signal as the graph offers: the person is explicitly recorded as heading that group, and the model has clearly picked up on it.

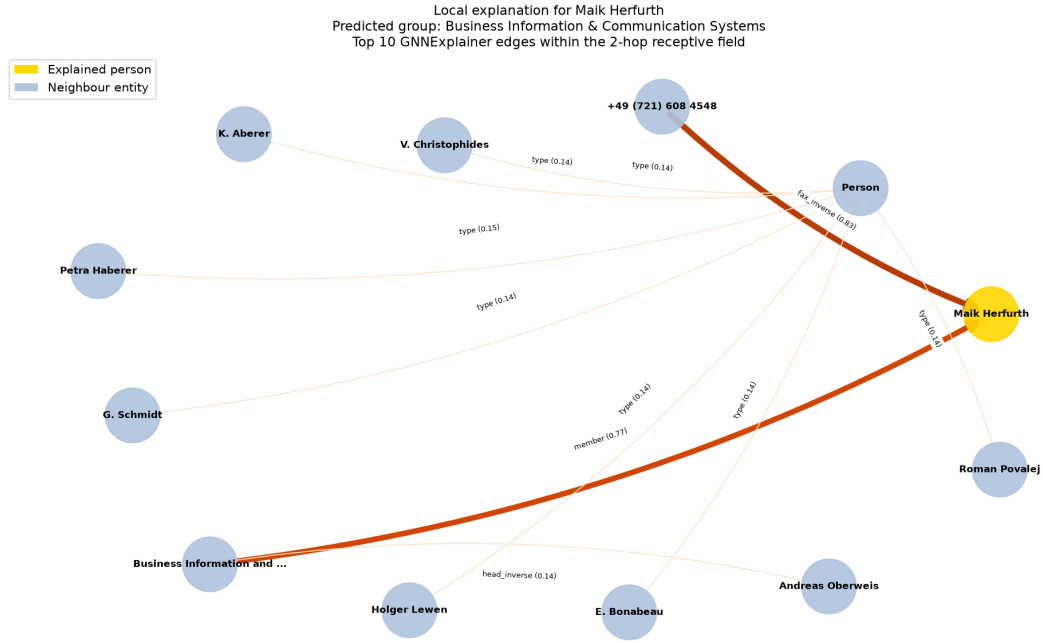


Figure 6: Local explanation subgraph for Maik Herfurth, predicted into Business Information & Communication Systems mainly through contact metadata and a `member` link.

For **Maik Herfurth**, predicted into the same group, the picture is similar but slightly less direct: the strongest edge is his fax number, followed by a plain `member` link from the group to him, plus a cluster of `type` edges tying several other people to the generic `Person` class. The metadata edges dominate here rather than an explicit leadership relation.

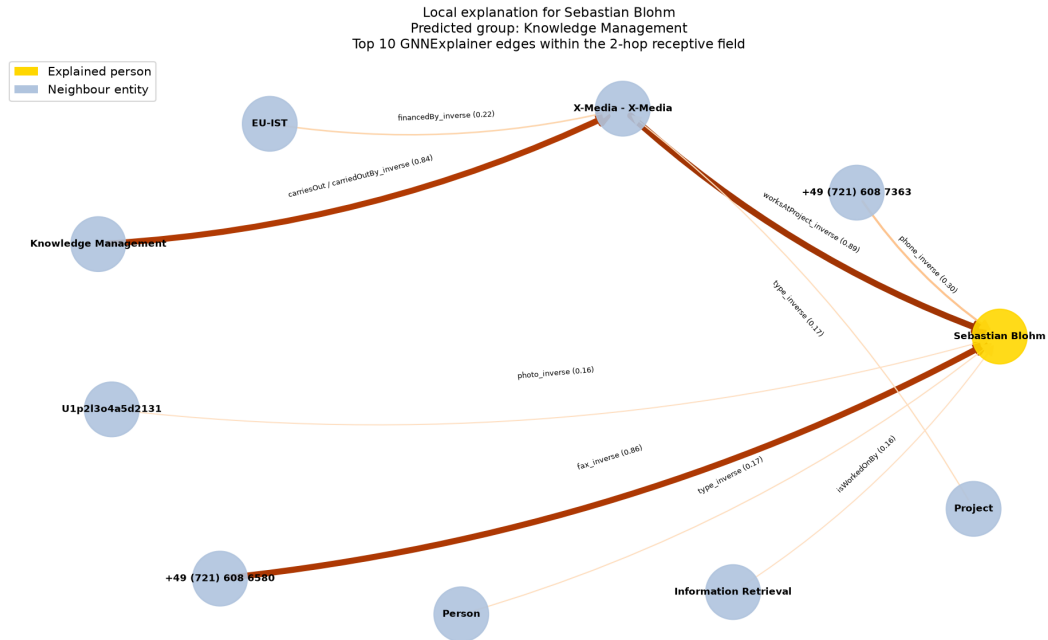


Figure 7: Local explanation subgraph for Sebastian Blohm, predicted into Knowledge Management largely through a shared project link.

For **Sebastian Blohm**, predicted as Knowledge Management, the top edge is an inverse `worksAtProject` link connecting him to the X-Media project, and the third-ranked edge is a `carriesOut` link from Knowledge Management to that same project. Together these two edges tell a coherent story: Blohm works on

a project that the Knowledge Management group runs, and the model is using that project as a bridge between the person and the correct label rather than relying purely on contact metadata.

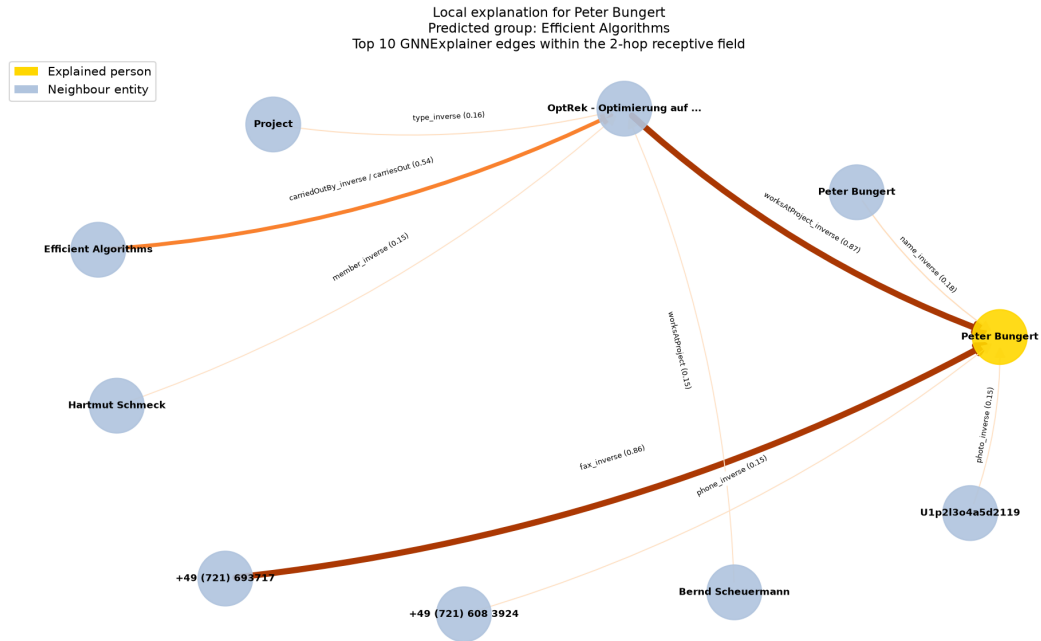


Figure 8: Local explanation subgraph for Peter Bungert, predicted into Efficient Algorithms through links to the OptRek project.

For **Peter Bungert**, predicted as Efficient Algorithms, the same pattern repeats with a different project: the strongest edge connects him to the OptRek project, and both a `carriedOutBy_inverse` and a `carriesOut` edge tie that project back to Efficient Algorithms. This is one of the cleaner explanations in the set, since the project genuinely is the one associated with his research group.

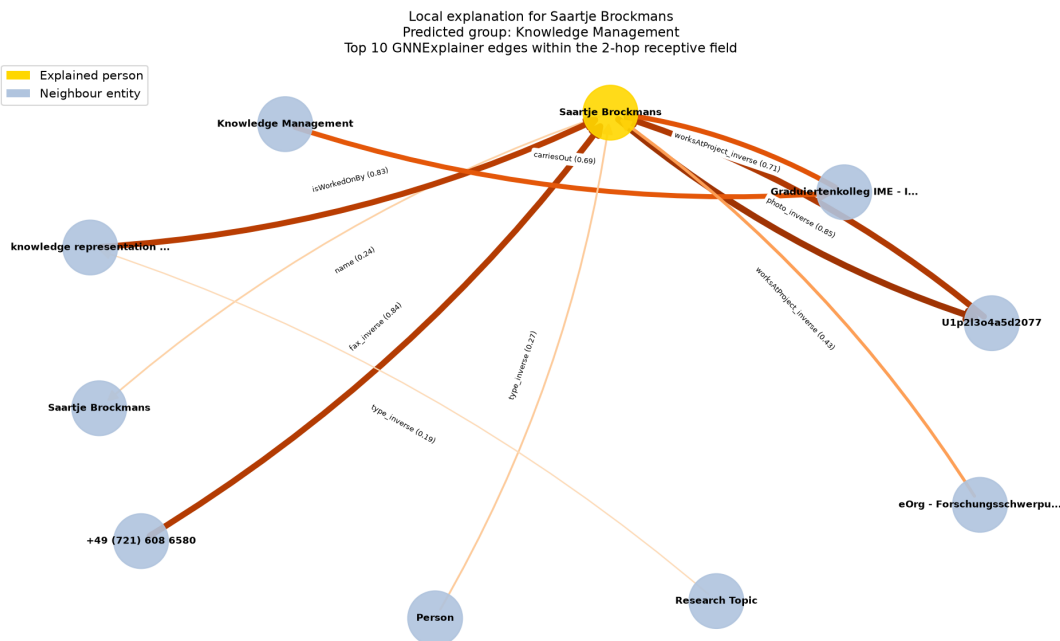


Figure 9: Local explanation subgraph for Saartje Brockmans, predicted into Knowledge Management via a mix of a photo reference and a research-topic link.

For **Saartje Brockmans**, also predicted as Knowledge Management, the top two edges are a photo reference in both directions, which is an odd but harmless artefact of the graph (a shared unique photo-file identifier acts as a stable structural link even though it carries no real semantic content), followed by her fax number and, more usefully, an **isWorkedOnBy** edge from “knowledge representation and reasoning” – a research topic that maps directly onto her actual group.

Taken together, the five local explanations show that the model does not lean on a single fixed rule. Sometimes it uses an explicit organisational edge such as **head** or **member**; other times it falls back on shared contact metadata such as fax or phone numbers; and in the cases that are arguably the most convincing (Blohm and Bungert), it uses project or topic links that connect the person to their group through genuine research activity rather than incidental metadata.

5 Evaluation

To quantitatively assess the quality of the generated explanations, we evaluated GNNExplainer across 20 correctly classified test nodes using four metrics:

- **Fidelity+**: Measures whether removing important edges changes the model’s prediction. Higher is better.
- **Fidelity-**: Measures whether keeping only important edges preserves the prediction. Higher is better.
- **Sparsity**: Fraction of edges not selected as important. Higher means more concise explanations.
- **Stability**: Cosine similarity between explanation masks across repeated runs. Higher means more consistent explanations.

Table 5: Quantitative evaluation of GNNExplainer over 20 test nodes.

Metric	Mean	Standard deviation
Fidelity+	0.4500	0.5104
Fidelity-	1.0000	0.0000
Sparsity	0.9999	0.0001
Stability	0.9999	0.00001

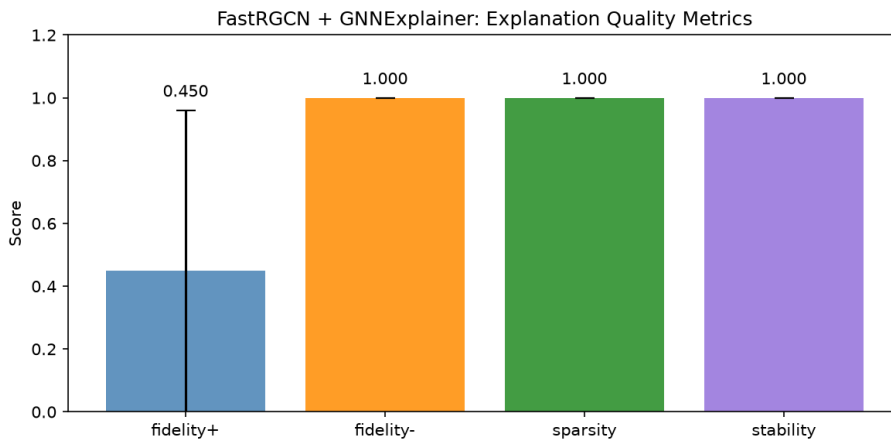


Figure 10: Explanation quality metrics averaged over 20 test nodes, with error bars showing one standard deviation.

The model got a perfect 1.0000 fidelity score across all 20 nodes. This is the strongest result. When we only keep the edges GNNExplainer flagged as important and drop everything else, the model still makes the exact same prediction it did with the full graph.

Sparsity and Stability are both maxed out. This means the explanations are tiny - usually less than 0.02% of edges pass the importance threshold. They're also super repeatable; running the explainer multiple times on the same node gives back the same results.

Fidelity+ is a bit more complicated. For 9 of the 20 nodes, removing those important edges changed the model's prediction. For the remaining 11, it didn't. This isn't necessarily the explainer failing. It probably just shows the graph is cluttered. In AIFB, a person can be tied to their research group via several different ways, like being a member directly, working on a shared project, or having contact info. So, even if you remove the top explanation, the model still has enough other evidence to reach the same conclusion.

6 Discussion

FastRGCN ended up being a reasonable fit for this task. It models AIFB's typed relational structure directly, it's quick to train on a dataset this size, and—unlike the plain RGCNConv layer—it's actually compatible with GNNExplainer, which is really what settled the decision for us. The explanation pipeline we built around it does tie predictions back to concrete graph evidence, and the local subgraphs for Blohm and Bungert especially show the kind of project-based reasoning we were hoping a relational model would pick up on.

That said, there are some real limitations worth being upfront about. The dataset is small and imbalanced, so we can't be too confident about performance on the minority Complexity Management class no matter which model we'd used. Several of the globally important relations—fax, phone, generic type edges—turn out to be metadata rather than actual academic signal, but that's a property of the AIFB graph itself, not something our explainer is inventing. And the mid-range Fidelity+ score is worth flagging too: just because the explainer flags an edge as important doesn't mean it's the only reason behind the prediction—in a densely connected graph like this, multiple independent pieces of evidence can end up pointing to the same answer.

7 Conclusion

For this part of the project, we built an explainable pipeline around a FastRGCN model with learnable node embeddings, training it across multiple seeds to predict research-group affiliation on the AIFB RDF dataset. We then ran GNNExplainer on a deterministically re-mapped version of the dataset to get both a global, relation-level explanation and five local explanations for individual people. When we evaluated this quantitatively on 20 test nodes, the results held up well: the selected explanation edges alone were enough to reproduce the model's original prediction every single time (Fidelity- = 1.0000), the explanations stayed sparse, and they were highly stable across repeated runs. About half of them also passed the stricter Fidelity+ test, which is a tougher bar to clear. What this tells us is that a relational GNN paired with an edge-mask explainer doesn't have to stay a black box—you can actually go in and check why it made a given prediction.

8 Contributions

Anmol Raj Srivastav implemented the deterministic dataset rebuild, the FastRGCN training loop (including the multi-seed evaluation), the GNNExplainer integration for both global and local explanations, and the quantitative explanation-evaluation script. Janhavi Palav contributed to result interpretation, data analysis pipeline, report writing, and the overall presentation of the findings. Both team members reviewed the final results together.

References

- [1] Ying, R., Bourgeois, D., You, J., Zitnik, M., Leskovec, J.: GNNExplainer: Generating Explanations for Graph Neural Networks. In: Advances in Neural Information Processing Systems 32 (2019).
- [2] Fey, M., Lenssen, J. E.: Fast Graph Representation Learning with PyTorch Geometric. In: ICLR Workshop on Representation Learning on Graphs and Manifolds (2019).

- [3] Schlichtkrull, M., Kipf, T. N., Bloem, P., van den Berg, R., Titov, I., Welling, M.: Modeling Relational Data with Graph Convolutional Networks. In: European Semantic Web Conference, pp. 593–607. Springer (2018).
- [4] AIFB RDF Dataset. https://figshare.com/articles/dataset/AIFB_DataSet/745364/1
- [5] PyTorch Geometric Entities Dataset Documentation. <https://pytorch-geometric.readthedocs.io/>
- [6] penAI ChatGPT was used exclusively for language editing, including paraphrasing, grammar, readability, and structural improvements.